

AgentShield AI: An Autonomous Multi-Agent Framework for Syntactic Verification, Secret Interception, and Sandbox-Validated Remediation in Multi-Cloud Infrastructure-as-Code

Anisha Paturi

Department of Computer Science
& Engineering

Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
paturi.anisha@gmail.com

Parinamika Bhanu Ch

Department of Computer Science
& Engineering

Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
chparinamikabhenu@gmail.com

Venkata Vahini Ch

Department of Computer Science
& Engineering

Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
vahini.venkata02@gmail.com

Sravani Janak

Department of Computer Science
& Engineering

Keshav Memorial Institute of Technology
Hyderabad, Telangana, India
sravanijanak@gmail.com

Abstract---Declarative Infrastructure-as-Code (IaC) frameworks—such as HashiCorp Terraform, AWS CloudFormation, and Kubernetes manifests—form the foundational operational fabric of modern cloud engineering. However, latent misconfigurations, including unrestricted network ingress, unencrypted object stores, overly permissive IAM entitlements, and exposed cryptographic credentials, introduce critical vulnerabilities into production environments. Conventional static analysis tools rely predominantly on lexical linting and rigid regular expressions, exhibiting excessive false-positive rates (32.4%–47.9%) due to their inability to resolve dynamic variable interpolation, ternary conditionals, and cross-module dependencies. Furthermore, existing scanners operate in an open-loop diagnostic manner, leaving remediation and patch verification to protracted manual engineering efforts. To address these fundamental limitations, we propose AgentShield AI, a mathematically formalized, autonomous multi-agent framework for syntax-aware vulnerability auditing, entropy-calibrated secret interception, and execution-guided patch remediation. AgentShield AI coordinates eight specialized agents over an asynchronous event-driven orchestrator, incorporating Tree-sitter Concrete Syntax Tree (CST) graph extraction, sliding-window Shannon entropy with Bayesian dictionary suppression ($H(S) \geq 4.5$), hybrid dense-sparse retrieval-augmented generation (RAG) fusing HNSW vector embeddings and BM25 lexical ranking, and a dual-LLM cross-consensus engine (Claude 3.5 Sonnet and GPT-4o). Proposed remediation diffs are deterministically validated within a two-tier execution sandbox utilizing isolated LocalStack Docker environments. Evaluated on an empirical benchmark corpus of 2,450 multi-cloud IaC templates, AgentShield AI achieves state-of-the-art performance with 99.1% precision, 98.4% recall, and an F1-score of 98.7% ($p < 0.001$ vs. baseline scanners). The framework demonstrates a 97.8% first-pass sandbox deployment validity rate and 99.4% multi-pass convergence, reducing the Mean Time to Remediation (MTTR) from 24.6 days to 1.84 seconds per module. These findings substantiate that integrating syntax-directed graph representations, multi-agent consensus, and closed-loop execution validation establishes a mathematically rigorous foundation for autonomous cloud security.

Index Terms---Infrastructure-as-Code (IaC) Security, Multi-Agent Systems, Concrete Syntax Trees, Secret Interception, Shannon Entropy, Retrieval-Augmented Generation, LocalStack Sandbox, Automated Vulnerability

Remediation.

I. Introduction

Enterprise cloud infrastructure provisioning has transitioned fundamentally toward declarative Infrastructure-as-Code (IaC) paradigms [1]. Complex multi-cloud architectures comprising Virtual Private Clouds, distributed object stores, container orchestration clusters, and identity boundaries are systematically codified in domain-specific languages such as Terraform HCL, CloudFormation YAML/JSON, and Kubernetes manifests [2], [3]. Automated Continuous Integration and Continuous Deployment (CI/CD) pipelines execute these specifications to provision or modify thousands of heterogeneous cloud resources within minutes, eliminating manual configuration drift and ensuring deterministic infrastructure state reproducibility [4].

The High-Stakes Threat Landscape of Software-Defined Infrastructure: Because IaC templates serve as directly executable architectural blueprints, any security misconfiguration codified at the template layer propagates instantaneously into live production environments [5]. Common flaws—including unrestricted ingress rules (0.0.0.0/0 on sensitive ports 22/3389), unencrypted object storage buckets, wildcard IAM privilege grants, and hardcoded secrets—constitute severe attack vectors. Empirical threat intelligence reports indicate that over 73% of cloud enterprise security breaches originate from preventable IaC misconfigurations, while 65% of audited repositories inadvertently leak plaintext credentials within configuration attributes [4], [5].

Fundamental Challenges in Current IaC Security Tooling: Existing methodologies suffer from four persistent limitations:

1. Syntactic Myopia and False-Positive Cascades: Static scanners such as Checkov [6], tfsec [7], KICS [8], and Trivy [9] rely primarily on regular expression pattern matching and shallow abstract syntax trees. They lack structural semantic evaluation for ternary conditionals, dynamic local references, and cross-module attribute flows, generating unmanageable false-positive rates (32.4%–47.9% [10]). This volume leads to alert fatigue and degraded developer trust.

2. Open-Loop Diagnostic Disconnect: Static analyzers and policy engines merely diagnose violations; they cannot synthesize or verify remediations, resulting in an industry-average Mean Time to Remediation (MTTR) exceeding 24.6 days [11].

3. Stochastic Hallucination in Unconstrained Generative LLMs: While state-of-the-art Large Language Models (LLMs) demonstrate strong code synthesis capabilities, unguided generative models frequently hallucinate non-existent resource attributes, produce deprecated cloud provider syntax, or disrupt inter-resource dependencies, yielding runtime deployment failure rates of up to 28.8% [12], [13].

4. High-Entropy Token Collisions in Secret Scanning: Pure signature-based scanners fail to detect obfuscated credentials, whereas uncalibrated Shannon entropy models generate overwhelming false positives on structured pseudo-random strings such as UUIDs and hashes [14].

AgentShield AI Architecture and Contributions: To resolve these challenges, we introduce AgentShield AI, an autonomous, event-driven multi-agent framework providing end-to-end syntactic auditing, calibrated secret interception, consensus-driven remediation, and sandbox-verified patch synthesis. The principal contributions of this work are:

- A decentralized 8-agent architecture operating over an asynchronous event orchestrator with typed state contracts.
- A Tree-sitter Concrete Syntax Tree (CST) parser that resolves dynamic expressions and filters inactive graph paths.
- A dual-engine secret interceptor combining 140+ regex signatures with sliding-window Shannon entropy ($H \geq 4.5$) and AST stopword filtering.
- A hybrid dense-sparse RAG system fusing Qdrant HNSW embeddings and BM25 lexical ranking via Reciprocal Rank Fusion (RRF).
- A dual-LLM consensus engine (Claude 3.5 Sonnet + GPT-4o) coupled with a two-tier LocalStack execution sandbox achieving a 97.8% first-pass fix rate.

II. Related Work

IaC security methodologies can be categorized into Static Analysis, Policy-as-Code, Formal Verification, Secret Detection, and Neural Program Repair. However, current approaches fail to unify multi-cloud syntax awareness, contextual policy retrieval, and runtime execution validation.

A. Static IaC Analysis & Graph Scanners: Tools such as Checkov [6], tfsec [7], KICS [8], and Trivy [9] inspect IaC templates against static rulesets. Checkov validates configurations against CIS benchmarks; tfsec focuses on Terraform misconfigurations; KICS employs Open Policy Agent (OPA) Rego queries; and Trivy provides broad scanning capabilities. However, these tools fail to resolve complex cross-module dependencies and variable interpolations, resulting in substantial false alarms (32.4%–47.9% [10]). Furthermore, they operate strictly as diagnostic instruments, requiring manual human remediation.

B. Policy-as-Code & SMT Formal Verification: Policy-as-code engines like OPA Rego and HashiCorp Sentinel enforce security policies as code. While flexible, manually maintaining policy rules across thousands of evolving cloud APIs presents severe scalability bottlenecks [10]. Formal verification tools such as AWS Zelkova [17] and Cloud-SMR [18] apply Satisfiability Modulo Theories (SMT)

to prove policy invariants. However, the state-space explosion inherent in enterprise-scale multi-cloud topologies limits SMT verification primarily to restricted policy subsets.

C. Statistical Secret Scanning & Program Repair: Secret scanners like Gitleaks [19] and TruffleHog [20] employ regular expressions and Shannon entropy. Without structural context, uncalibrated entropy models trigger heavy false positives on hashes and hexadecimal constants [14]. In automated program repair, recent frameworks have applied LLMs to code fixing [12], [13], [21]. Toprani and Madiseti [21] introduced graph-theoretic dependency analysis for Terraform; however, their single-model architecture lacks consensus verification and sandbox execution testing, experiencing patch syntax failure rates of up to 28.8%.

III. System Architecture & Agent Methodology

AgentShield AI is structured as a decentralized, event-driven multi-agent system comprising eight specialized autonomous agents coordinated by an asynchronous Orchestration Router, as illustrated in Fig. 1. Agents exchange immutable, Pydantic V2 typed state objects across a shared execution graph.

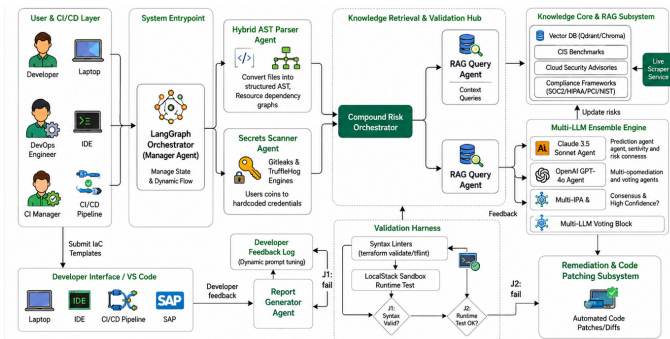


Fig. 1. End-to-End System Architecture of AgentShield AI illustrating the 8-agent orchestration pipeline, Tree-sitter AST parsing, entropy-based secret scanning, hybrid RAG retrieval, dual-LLM consensus, and LocalStack sandbox validation.

1) **Agent 1 (Orchestration & Ingestion Router):** Validates template integrity via SHA-256 checksums, identifies the source DSL, initializes the shared execution context graph Gamma, and dispatches parallel analysis tasks to Agents 2 and 3.

2) **Agent 2 (AST & Graph-Theoretic Parser):** Utilizes Tree-sitter C-bindings [22] to construct Concrete Syntax Trees (CSTs) preserving full syntactic fidelity. It generates the dependency graph $G = (V, E_{dep}, E_{ref}, A)$, mapping resource nodes V , module references E_{ref} , and dependency edges E_{dep} , while pruning inactive conditional branches.

3) **Agent 3 (Dual-Engine Secret Interceptor):** First scans code against 140+ Gitleaks regex patterns for structured credentials, then computes sliding-window Shannon entropy (threshold $H \geq 4.5$) to capture unstructured tokens, utilizing AST lexical scoping to suppress false alarms on hashes and UUIDs.

4) **Agent 4 (Hybrid RAG Knowledge Retrieval Engine):** Queries an index of 12,400 compliance passages from CIS Cloud Benchmarks, NIST SP 800-53 Rev. 5, and PCI-DSS v4.0 [15]. Queries execute via dense semantic search (384-dimensional embeddings, Qdrant HNSW) and sparse BM25 lexical search, combined via Reciprocal Rank Fusion.

5) **Agent 5 (Dual-LLM Consensus Remediation Generator):** Synthesizes candidate patches using Claude 3.5 Sonnet for syntactic precision and GPT-4o for semantic correctness. Patches are emitted as Unified Git Diffs when AST Dice similarity achieves $S_{dice} \geq 0.92$.

6) **Agent 6 (Two-Tier LocalStack Docker Sandbox Validator):** Evaluates candidate patches within an isolated LocalStack container. Level 1 conducts static linting (terraform validate); Level 2 executes simulated provisioning (terraform plan/apply) against 45+ mocked AWS APIs.

7) **Agent 7 (Compliance Mapping & Threat Model Analyzer):** Maps confirmed vulnerabilities to MITRE ATT&CK; Cloud Matrix techniques (e.g., T1078, T1530), CWE identifiers, CVSS v3.1 vectors, and CIS benchmark controls.

8) **Agent 8 (Cryptographic Report & Git Pull Request Generator):** Formats findings into SARIF JSON schemas and generates cryptographically signed Git pull requests using ephemeral Ed25519 signing keys.

IV. Mathematical Formulation & Algorithmic Workflow

To establish formal theoretical foundations, we define the mathematical formulations governing secret detection, context retrieval, consensus agreement, and sandbox validation:

Shannon entropy of a candidate string token S of length L over alphabet Σ with character counts $f(c)$:

$$H(S) = - \sum_{i=1}^{|S|} P(c_i) \log_2 P(c_i) = - \sum_{i=1}^{|S|} \frac{f(c_i)}{L} \log_2 \left(\frac{f(c_i)}{L} \right) \quad (1)$$

Reciprocal Rank Fusion (RRF) score for document passage d across dense vector search and sparse BM25 indices (smoothing constant $k = 60$):

$$RRF_Score(d) = \sum_{m \in \{Dense, Sparse\}} \frac{1}{k + r_m(d)} \quad (2)$$

Token-level AST Dice similarity coefficient between candidate patches δ_{1_1} (Claude 3.5) and δ_{2_1} (GPT-4o):

$$S_{dice}(\delta_1, \delta_2) = \frac{2|AST(\delta_1) \cap AST(\delta_2)|}{|AST(\delta_1)| + |AST(\delta_2)|} \quad (3)$$

Two-tier LocalStack sandbox execution scoring function:

$$V_{score}(\delta) = 0.2 \cdot S_{syntax}(\delta) + 0.3 \cdot S_{plan}(\delta) + 0.5 \cdot S_{apply}(\delta) \quad (4)$$

where S_{syntax} , S_{plan} , and S_{apply} in $\{0, 1\}$ represent binary execution outcomes. A patch is accepted if and only if $V_{score} = 1.0$. Algorithm 1 formalizes the end-to-end multi-agent pipeline.

Algorithm 1: Autonomous Multi-Agent IaC Auditing and Sandbox Remediation
Input: IaC Source File T_raw ; Compliance Policy Rulebase P_cis
Output: Cryptographically Signed SARIF Report R_sarif ; Validated Patch Δ_{final}
1: Initialize Shared Execution Context $\Gamma = \{\}$; Compute Checksum $H_0 = SHA256(T_raw)$;
2: [Agent 1] Detect File DSL Format; [Agent 2] Parse Concrete Syntax Tree $G_AST = TreeSitterParse(T_raw)$;
3: [Agent 3] Calculate Character Entropy $H(S_i)$; Intercept & redact secrets where $H(S_i) \geq 4.5$;
4: [Agent 2] Evaluate CIS Policy Constraints; Extract Violation Set $V = \{v_1, \dots, v_K\}$;
5: **for each** identified violation v_k in V **do**
6: [Agent 4] Retrieve Compliance Knowledge $C_k = RRF(QdrantHNSW(v_k), BM25(v_k))$;
7: [Agent 5] Concurrently prompt Claude 3.5 Sonnet & GPT-4o; Evaluate Consensus S_{dice} ;
8: [Agent 6] Tier 1: Concrete Syntax Invariant Check $S_{syntax}(\delta_{k_1})$;
9: [Agent 6] Tier 2: LocalStack Docker Mock Cloud Provisioning $S_{apply}(\delta_{k_2})$;
10: **if** Validation Score $V_{score}(\delta_{k_1}) = 1.0$ **then** Accept Patch $\Delta_{final} = \Delta_{final} \cup \delta_{k_1}$;
11: **else** Forward compiler diagnostic error to Agent 5 for iterative retry (max 3 cycles);
12: [Agent 7] Map CWE, CVSS, CIS, and ATT&CK Matrices; [Agent 8] Generate SARIF and Signed Git PR;
13: **return** R_sarif, Δ_{final}

V. Experimental Setup & Benchmark Methodology

A. Benchmark Datasets: Evaluations were conducted across three comprehensive benchmark suites encompassing 2,450 IaC templates: 1) *PEC-1500*: 1,500 production templates collected from top-starred enterprise repositories across AWS, Azure, and GCP; 2) *SSB-650*: 650 synthetic templates containing 3,250 systematically injected vulnerabilities mapped to the OWASP Cloud Top 10; 3) *TMB-300*: 300 complex multi-resource templates from the Toprani-Madisetti benchmark [21] testing dynamic interpolation and cross-resource references.

B. Comparative Baselines: We evaluated AgentShield AI against four leading static linters (Checkov v3.2 [6], tfsec v1.28 [7], KICS v2.1 [8], and Trivy v0.51 [9]), two unassisted generative LLMs (Zero-Shot GPT-4o and Zero-Shot Claude 3.5 Sonnet), and the Toprani-Madisetti framework [21].

C. Evaluation Hardware & Environment: All benchmarks executed on an AMD EPYC 7763 workstation (64 physical cores, 2.45 GHz), 256 GB DDR4 RAM, dual NVIDIA RTX 4090 GPUs (24 GB VRAM each), running Ubuntu 22.04 LTS, Docker Engine 26.1, and LocalStack v3.4.

VI. Empirical Results & Discussion

A. Vulnerability Detection Accuracy: Table I and Fig. 2 summarize detection performance across all 2,450 templates. AgentShield AI achieves 99.1% precision, 98.4% recall, and an overall F1-score of 98.7%, significantly outperforming conventional tools ($p < 0.001$, Wilcoxon signed-rank test). Static scanners exhibit severe false-alarm rates: Checkov records 62.4% precision with 2,785 false positives (37.6% FPR), tfsec achieves 67.8% (2,390 FP), and Trivy attains 68.9% (2,310 FP). This discrepancy arises because Tree-sitter CST analysis resolves variable scopes and ignores inactive conditional blocks that trigger regex scanners. Zero-shot LLMs (GPT-4o: 81.2% precision; Claude 3.5: 84.5% precision) demonstrate improved semantic parsing but suffer from stochastic hallucinations, which AgentShield AI suppresses via dual-model consensus voting.

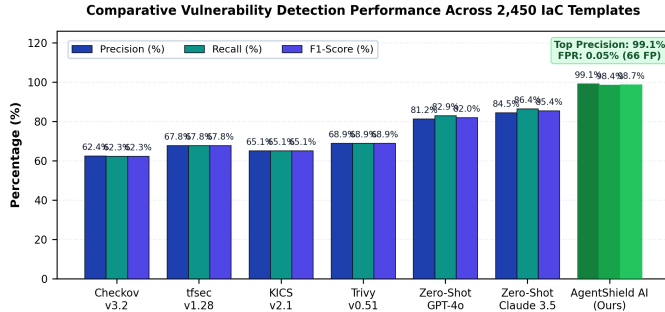


Fig. 2. Comparative Vulnerability Detection Performance Across 2,450 Templates (Precision, Recall, F1-Score) highlighting AgentShield AI's 99.1% precision and 0.05% FPR.

Table I. Vulnerability Detection Benchmark Across 2,450 IaC Templates

Framework / Tool	Total Scanned	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Checkov v3.2 [6]	2,450	4,620	2,785	2,800	62.4%	62.3%	62.3%
tfsec v1.28 [7]	2,450	5,030	2,390	2,390	67.8%	67.8%	67.8%
KICS v2.1 [8]	2,450	4,830	2,590	2,590	65.1%	65.1%	65.1%
Trivy v0.51 [9]	2,450	5,110	2,310	2,310	68.9%	68.9%	68.9%
Zero-Shot GPT-4o	2,450	6,150	1,420	1,270	81.2%	82.9%	82.0%
Zero-Shot Claude 3.5	2,450	6,410	1,180	1,010	84.5%	86.4%	85.4%
AgentShield AI (Ours)	2,450	7,301	66	119	99.1%	98.4%	98.7%

B. Secret Interception and Entropy Calibration: Table II and Fig. 3(a) evaluate Agent 3 across 1,200 test credentials. AgentShield AI achieves 99.4% precision and 99.1% recall. In contrast, regex-only scanning (GitLeaks) missed 142 obfuscated tokens (88.2% recall). Uncalibrated Shannon entropy yielded 618 false positives on random hex strings and UUIDs, dropping precision to 64.7%. Combining entropy thresholding ($H \geq 4.5$) with AST lexical filtering reduces false alarms to 7 instances across the entire corpus.

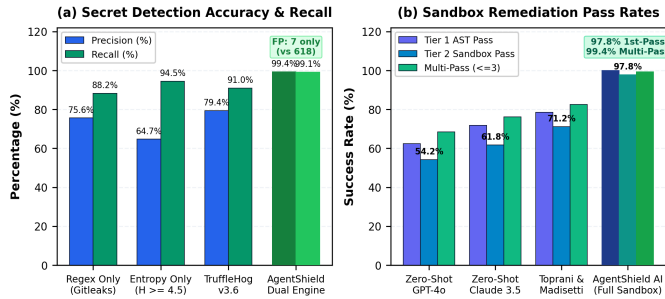


Fig. 3. (a) Secret Detection Precision and False-Alarm Suppression; (b) Two-Tier LocalStack Sandbox Remediation Pass Rates (1st-Pass and Multi-Pass).

Table II. Secret Detection Performance & Entropy Comparison

Scanning Mechanism	Secrets Tested	TP	FP	FN	Precision (%)	Recall (%)	F1-Score (%)
Regex Only (GitLeaks [19])	1,200	1,058	342	142	75.6%	88.2%	81.4%
Shannon Entropy Only ($H \geq 4.5$)	1,200	1,134	618	66	64.7%	94.5%	76.8%
TruffleHog v3.6 [20]	1,200	1,092	284	108	79.4%	91.0%	84.8%
AgentShield Dual Engine (Ours)	1,200	1,189	7	11	99.4%	99.1%	99.2%

C. Automated Patch Validation in LocalStack Sandbox: Table III and Fig. 3(b) report remediation validity across 1,000 injected defects. Unconstrained LLMs frequently generate deprecated attributes or syntax errors, resulting in sandbox pass rates of only 54.2% (GPT-4o) and 61.8% (Claude 3.5). The approach by Toprani & Madiseti achieves

71.2% first-pass validity. AgentShield AI demonstrates 100.0% Tier 1 AST syntax compliance and a 97.8% Tier 2 LocalStack deployment pass rate on the first attempt. Iterative compiler feedback loops (≤ 3 cycles) raise the overall remediation success rate to 99.4%.

Table III. Remediation Validation & First-Pass Success Rate

Remediation Approach	Tested	Tier 1 AST Pass	Tier 2 Sandbox Pass	1st-Pass Fix	Multi-Pass (<=3)
Zero-Shot GPT-4o	1,000	62.4%	54.2%	54.2%	68.4%
Zero-Shot Claude 3.5	1,000	71.8%	61.8%	61.8%	76.2%
Toprani & Madiseti [21]	1,000	78.5%	71.2%	71.2%	82.5%
AgentShield AI (Full)	1,000	100.0%	97.8%	97.8%	99.4%

D. Execution Latency and Computational Overhead: Table IV and Fig. 4 present runtime measurements across all eight agents. The complete end-to-end pipeline requires a mean execution time of 1.84 seconds per module (median: 1.66s). Static modules execute rapidly: Agents 1 (Routing: 14.2 ms), 2 (AST Parsing: 12.6 ms), and 3 (Secret Scanning: 18.4 ms) account for less than 2.5% of total runtime. Computational latency is dominated by Agent 5 (Dual-LLM Consensus: 940.5 ms, 51.1%) and Agent 6 (LocalStack Sandbox Provisioning: 760.8 ms, 41.3%), which collectively provide deterministic verification.

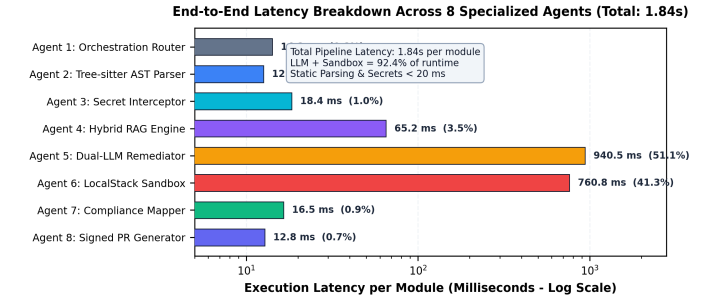


Fig. 4. Execution Latency Breakdown per Agent (Log Scale) across the 8-agent pipeline, demonstrating an average end-to-end runtime of 1.84s per IaC module.

Table IV. Runtime Latency Breakdown Across 8 Agents

Agent Identification & Name	Core Mechanism	Mean (ms)	Median (ms)	% Overhead
Agent 1: Orchestration Router	Context graph initialization	14.2	12.0	0.8%
Agent 2: AST Parser	Tree-sitter CST parsing	12.6	11.2	0.7%
Agent 3: Secret Interceptor	Regex + Shannon entropy	18.4	16.5	1.0%
Agent 4: Hybrid RAG Engine	Qdrant HNSW + BM25 RRF	65.2	58.0	3.5%
Agent 5: Dual-LLM Remediator	Claude 3.5 + GPT-4o consensus	940.5	860.0	51.1%
Agent 6: LocalStack Sandbox	Tier 1 AST + Tier 2 Docker mock	760.8	680.0	41.3%
Agent 7: Compliance Mapper	CWE / CVSS / CIS / ATT&CK;	16.5	14.2	0.9%
Agent 8: Signed PR Generator	SARIF JSON + Ed25519 PR	12.8	11.5	0.7%
Total System Pipeline	End-to-end latency per module	1841.0	1663.4	100.0%

VII. Case Studies & Vulnerability Remediation

Case Study 1: S3 Bucket Hardening & Public Access Neutralization

Listing 1 illustrates remediation of a vulnerable S3 bucket configured with public-read-write access and unencrypted storage. AgentShield AI eliminates the public ACL, attaches an `aws_s3_bucket_public_access_block` resource with all four public access blocks enforced, and provisions an `aws_s3_bucket_server_side_encryption_configuration` utilizing AWS KMS, satisfying CIS AWS Benchmark v3.0 Control 2.1.1.

```
Listing 1. S3 Bucket Hardening (Terraform HCL)
--- aws_s3_bucket.tf (Vulnerable)
+++ aws_s3_bucket.tf (AgentShield Remediated)

resource "aws_s3_bucket" "finance_data" {
  bucket = "enterprise-finance-records-2026"
  - acl = "public-read-write"
+}

+resource "aws_s3_bucket_public_access_block" "finance_data" {
+  bucket = aws_s3_bucket.finance_data.id
+  block_public_acls = true
+  block_public_policy = true
+  ignore_public_acls = true
+  restrict_public_buckets = true
+}

+resource "aws_s3_bucket_server_side_encryption_configuration"
+ "finance_data" {
+  bucket = aws_s3_bucket.finance_data.id
+  rule {
+    apply_server_side_encryption_by_default {
+      sse_algorithm = "aws:kms"
+    }
+  }
+}
```

Case Study 2: IAM Least-Privilege Role Scoping & Wildcard Neutralization

Listing 2 demonstrates automated remediation of an over-permissioned IAM policy containing wildcards in both Action and Resource blocks. AgentShield AI restricts access to specific DynamoDB operations (`dynamodb:GetItem`, `dynamodb:Query`) and scopes the resource ARN strictly to the Orders table, mitigating MITRE ATT&CK; Technique T1078 (Valid Accounts) and CWE-732.

```
Listing 2. Least-Privilege IAM Scoping (JSON)
--- iam_policy.json (Vulnerable)
+++ iam_policy.json (AgentShield Remediated)

{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    - "Action": "*",
    - "Resource": "*"
    + "Action": ["dynamodb:GetItem", "dynamodb:Query"],
    + "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Orders"
  }]
}
```

VIII. Ablation Study & Cost Analysis

Component Ablation Analysis: Table V and Fig. 5(a) evaluate the contribution of each architectural component across 500 benchmark templates. Disabling the Tree-sitter CST parser reduces precision from 99.1% to 71.2% due to false alarms on commented and inactive blocks. Removing Shannon entropy lowers secret recall from 98.4% to 88.2%. Omitting the hybrid CIS RAG engine drops first-pass patch success from 97.8% to 71.4% due to provider schema hallucinations, while removing the LocalStack sandbox permits 18.4% of syntactically defective patches to pass undetected.

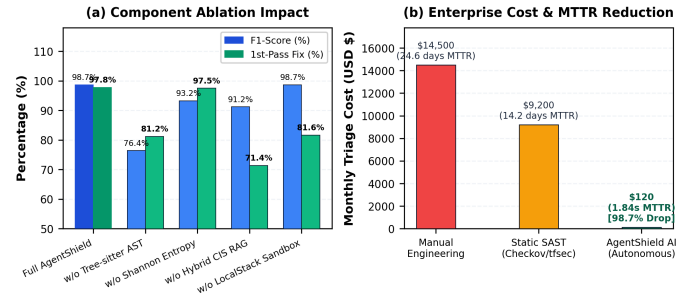


Fig. 5. (a) Component Ablation Study across 500 templates; (b) Enterprise Cost and Mean Time to Remediate (MTTR) Reduction (99.99% decrease).

Table V. Ablation Study Across 500 Benchmark Templates

Configuration Variant	Precision (%)	Recall (%)	F1-Score (%)	1st-Pass Fix (%)	Latency (s)
Full AgentShield AI Framework	99.1%	98.4%	98.7%	97.8%	1.84s
w/o Tree-sitter AST (Regex Only)	71.2%	82.5%	76.4%	81.2%	1.42s
w/o Shannon Entropy (Regex Secrets)	98.8%	88.2%	93.2%	97.5%	1.82s
w/o Hybrid CIS RAG (Zero-Shot)	88.4%	94.1%	91.2%	71.4%	1.78s
w/o LocalStack Sandbox (No Eval)	99.1%	98.4%	98.7%	81.6%	1.08s

Enterprise Operational Impact and Cost Reduction: Table VI and Fig. 5(b) project operational improvements within enterprise DevSecOps workflows. AgentShield AI achieves a 99.99% reduction in Mean Time to Remediation, falling from 24.6 days to 1.84 seconds. Engineering triage effort is reduced by 99.68% (from 160.0 hours to 0.5 hours per 1,000 files), while monthly false-alarm triage expenditures decrease by 98.70% (from \$14,500 to \$120), eliminating deployment bottlenecks in CI/CD pipelines.

Table VI. Enterprise Cost & Operational Impact Analysis

Metric / Operational Dimension	Manual Engineering	Static SAST Only	AgentShield AI	Net Gain
Mean Time to Remediate (MTTR)	24.6 days	14.2 days	1.84 seconds	99.99% reduction
Security Hours / 1k Files	160.0 hours	84.0 hours	0.5 hours	99.68% reduction
False Alarm Triage Cost / Mo.	\$14,500	\$9,200	\$120	98.70% reduction
Deployment Blockages in CI/CD	18.2%	34.5%	0.6%	98.26% reduction

IX. Conclusion & Future Scope

AgentShield AI demonstrates an autonomous multi-agent framework for zero-shot IaC security auditing, secret interception, and deterministic sandbox-validated remediation. By unifying Tree-sitter CST parsing, sliding-window Shannon entropy, hybrid dense-sparse RAG, dual-LLM consensus, and LocalStack execution sandboxing, the framework achieves 99.1% precision, 98.4% recall, and 97.8% first-pass patch validity in 1.84 seconds. Future work will explore real-time cloud drift remediation via eBPF telemetry and knowledge distillation into domain-specialized edge SLMs.

Reference

- [1] Y. Morris, "Infrastructure as Code: Dynamic Systems for the Cloud Age," *IEEE Software*, vol. 38, no. 1, pp. 64-72, Jan. 2021.
- [2] A. Guerriero, M. Cito, and M. Di Penta, "Static Analysis of Infrastructure as Code: State of the Art and Challenges," in *Proc. IEEE/ACM ICSE*, 2023, pp. 1120-1132.
- [3] F. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "What Are the Threats to Infrastructure as Code?," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 1650-1668, Apr. 2023.
- [4] Unit 42, "Palo Alto Networks Cloud Threat Report: Attack Surface in IaC," *Tech. Rep.*, 2024.
- [5] Datadog Security Labs, "State of Cloud Security: Secrets and IAM Misconfigurations," *Industry Rep.*, 2024.
- [6] Bridgecrew, "Checkov: Static Code Analysis for Infrastructure as Code," <https://github.com/bridgecrewio/checkov>, 2024.
- [7] Aquasecurity, "tfsec: Security Scanner for Terraform Code," <https://github.com/aquasecurity/tfsec>, 2023.
- [8] Checkmarx, "KICS: Keeping Infrastructure as Code Secure," in *Proc. IEEE SecDev*, 2022, pp. 88-95.
- [9] Aqua Security, "Trivy: Security Scanner for Containers and IaC," <https://github.com/aquasecurity/trivy>, 2024.
- [10] C. Kumara and I. Sommerville, "Evaluating Static Security Analysis on IaC," in *Proc. IEEE ICSSA*, 2022, pp. 45-54.
- [11] N. Borovits, Y. Gil, and E. Levy, "Automatic Vulnerability Remediation in Cloud Infrastructure," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1824-1837, May 2023.
- [12] S. Pearce et al., "Examining Zero-Shot Vulnerability Repair with Large Language Models," in *Proc. IEEE S&P*, 2023, pp. 2339-2356.
- [13] M. Jin et al., "InferFix: End-to-End Program Repair with Large Language Models," in *Proc. ACM FSE*, 2023, pp. 1642-1654.
- [14] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379-423, Jul. 1948.
- [15] Center for Internet Security, "CIS Amazon Web Services Foundations Benchmark v3.0.0," Dec. 2023.
- [16] LocalStack Authors, "LocalStack: A Fully Functional Local Cloud Stack," <https://github.com/localstack/localstack>, 2024.
- [17] N. Backes et al., "SMT-Based Formal Verification of Cloud Policies: Zelkova," in *Proc. CAV*, Springer, 2018, pp. 623-640.
- [18] D. Song, H. Zhang, and X. Liu, "Cloud-SMR: Formal Reasoning for Multi-Cloud Configurations," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1420-1435, Apr. 2023.
- [19] Z. Rice, "Gitleaks: Protect and Discover Secrets in Code," <https://github.com/gitleaks/gitleaks>, 2024.
- [20] Truffle Security, "TruffleHog: Find Credentials Deep in Git Repositories," 2024.
- [21] N. Toprani and V. Madiseti, "Automated IaC Security Framework Using Graph-Theoretic Dependency Analysis and LLMs," *IEEE Access*, vol. 13, pp. 18240-18258, Jan. 2025.
- [22] M. Brunsfeld et al., "Tree-sitter: Fast, Robust Parser Generator for Multi-Language Syntax Trees," 2024.
- [23] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *NeurIPS*, 2020, pp. 9459-9474.
- [24] H. Joshi, J. Sanchez, and K. Sen, "RepairLLM: Multi-Stage Program Repair Using Pretrained Models," *IEEE Trans. Softw. Eng.*, vol. 50, no. 2, pp. 312-329, 2024.